

Chapitre 3 – Les arbres et les arborescences

3.1. Introduction

Dans ce chapitre, nous allons considérer uniquement les graphes simples, donc les graphes sans boucle et qui comportent au maximum une seule arête/arc entre 2 sommets. Des graphes que nous appellerons ici $G = (V, E)$, et ayant n sommets et m arêtes/arcs. Deux notions importantes ont besoin d'être rappelées :

- *La notion de connexité* : G est connexe si pour toute paire de sommets, il existe une chaîne/chemin entre eux.
- *La notion de cycle* : G est cyclique s'il contient une chaîne fermée $x_0, x_1, x_2, \dots, x_0$. (il est dit acyclique dans le cas contraire).

Nous rappelons également que :

- G est connexe $\Rightarrow m \geq n-1$
- G est acyclique $\Rightarrow m \leq n-1$

3.2. Définitions générales et propriétés

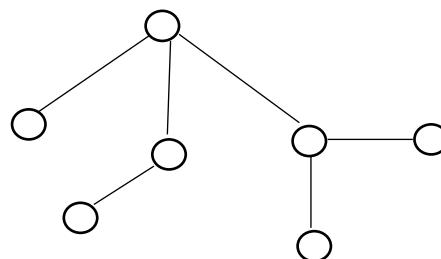
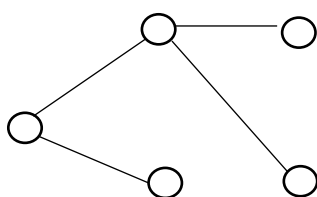
a) Arbre

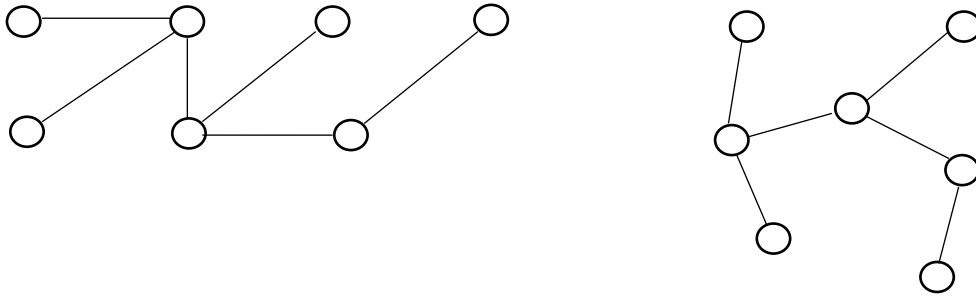
Définition 1 : Un *arbre* est un graphe

- *non-orienté*
- *connexe et*
- *sans cycle.*

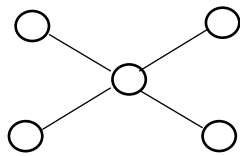
Si l'arbre est d'ordre n , alors $m = n-1$

Exemples : Les graphes suivants sont des arbres

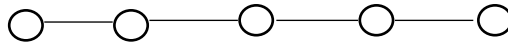




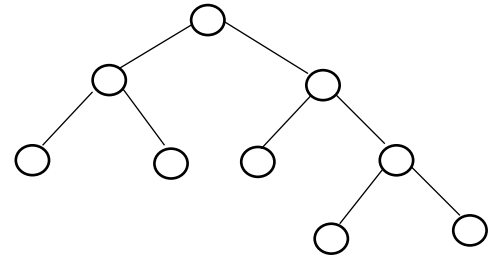
Exemples d'arbres particulier



Arbre étoile



Arbre linéaire/chemin



Arbre binaire

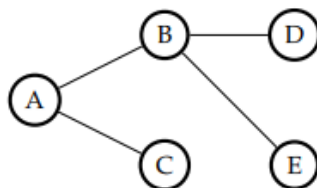
Théorème très classique dans le monde des arbres (vous pouvez chercher la démonstration sur le net)

Pour un arbre G non orienté d'ordre n ($n > 2$), les assertions suivantes sont équivalentes :

1. G est connexe et sans cycle (donc un arbre);
2. G est connexe et minimal : si on supprime une arête quelconque, il ne sera plus connexe ;
3. G est connexe et $m = n - 1$
4. G est sans cycle et maximal : si on lui rajoute encore une arête alors il contiendra un cycle ;
5. G est sans cycle et $m = n - 1$
6. Il existe une chaîne unique entre deux sommets quelconques de G .

Donc, il suffit de démontrer l'une de ces assertions pour démontrer que le graphe est un arbre !

Un arbre peut être colorié avec deux couleurs, il est donc biparti. Tout sommet pendant d'un arbre s'appelle feuille. Dans un arbre d'ordre $n \geq 2$, il existe au moins deux sommets pendants (donc deux feuilles). Dans le graphe : C, D et E: sont des feuilles

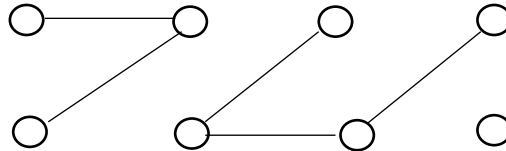


b) Forêt

Définition 2 : Un graphe non connexe et dont chaque composante connexe est un arbre est appelé **forêt**.

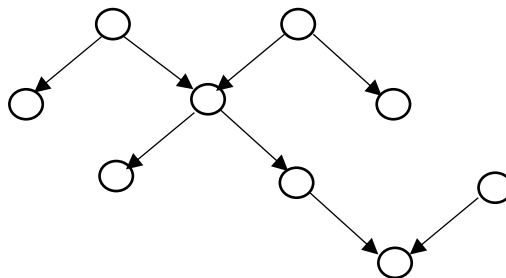
Une forêt est un ensemble disjoint d'arbres

Exemple : Le graphe suivant est une forêt d'arbres

**c) Polyarbre**

Définition 3 : un polyarbre est un arbre orienté, arbre dirigé. Sa définition peut être plus simplifiée : si on remplace tous les arcs par des arêtes, nous obtenons un arbre (connexe et sans cycle).

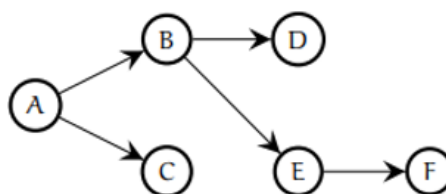
Exemple : le graphe suivant est un polyarbre

**d) Racine, anti-racine**

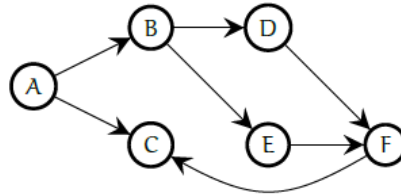
Définition 4 : Dans un graphe orienté, on appelle une **racine**, un sommet permettant de rejoindre n'importe quel autre sommet

Une **anti-racine** est le concept opposé de la racine.

Exemple : Dans le graphe suivant, le sommet A est une racine,



Si on ajoute un arc du sommet D au sommet F et un arc du sommet F au sommet C dans le graphe G, C devient une anti-racine.



e) Arborescence

Définition 5 : Une *arborescence* est un graphe orienté G sans circuit tel que :

1. G possède un sommet racine S ;
2. Pour tout sommet x différent de S, il existe un chemin unique de S vers x, donc $d^-(x) = 1$ (il a un seul prédécesseur)

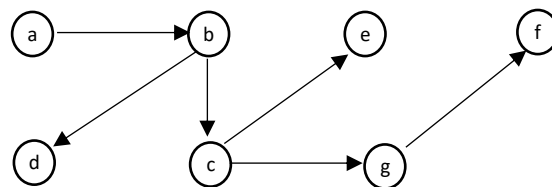
Nous pouvons donner aussi une autre définition pour une arborescence :

Définition 6 : Une arborescence est un graphe orienté acyclique, comportant un sommet particulier qui permet d'atteindre tous les autres sommets et dont le graphe non orienté sous-jacent est un arbre

Remarque 1 : Toute arborescence est un polyarbre mais l'inverse est faux.

Remarque 2 : si l'arborescence est d'ordre n, alors elle contient $m = n-1$

Exemple : Le graphe suivant est une arborescence de racine a



f) Anti-Arborescence

Définition 7 : Une anti-arborescence est un arbre orienté où tout point possède au plus un successeur. Si on inverse le sens des arcs d'une arborescence, on obtient une anti-arborescence. Un seul point ne possède pas de successeurs, il est appelé anti-racine

3.3. Les applications des arbres/arborescences

Parmi les applications des arborescences : Découpage hiérarchique comme :

- le découpage d'un livre en chapitres, sections, paragraphes, etc.
- Liens généalogiques, par exemple dans une famille, où le modèle a été repris en informatique sous la forme de l'héritage en conception orientée objet.
 - les sommets représentent les membres
 - et les arcs les liens de parenté.

Beaucoup d'algorithmes utilisent les arbres pour résoudre des problèmes de manière plus efficace

3.4. Arbre de couverture

Soit $G = (X, E)$ un graphe non-orienté connexe.

Définition 5 : Un *arbre couvrant* est un graphe *partiel* de G qui :

- Est un *arbre* donc, connexe et acyclique
- *Partiel* : touche tous les sommets de G , avec un sous ensemble de ses arêtes

On dit également : **arbre de couverture**

Remarque importante : Dans un graphe, on peut construire **plusieurs arbres couvrants**, leur nombre (AC) dépend de la forme du graphe. Pour les graphes complets et biparti complet, nous donnons les formules suivantes pour calculer AC

- Graphe complet K_n : $AC = n^{n-2}$
- Biparti complet $K_{m,n}$: $AC = m^{n-1} * n^{m-1}$

Théorème : $G=(V,E)$ est connexe est équivalent à dire G contient un arbre couvrant

3.5. Arbre de couvrant de poids minimal

Dans cette section, nous allons considérer les graphes pondérés, où à chaque arête est associée un poids.

Définition 6 : L'arbre couvrant de poids minimal ou l'arbre couvrant minimal (ACM) est l'arbre couvrant dont la somme des poids des arêtes est minimale

Remarque : Si le graphe *n'est pas connexe*, alors on cherche un ACM pour chaque composante connexe. Et donc une *forêt* de poids minimal.

3.6. Les applications des ACM

La construction des ACM a plusieurs applications.

- Exemple 1 : Dans un système de communication par message, si un nœud souhaite diffuser un message, il peut construire un arbre de couverture de poids minimal pour faire parvenir le message à tous les autres nœuds avec un coût minimal.
- Exemple 2 : Dans un réseau, par exemple un réseau d'ordinateurs, on utilise des arbres pour effectuer des connexions si on veut minimiser le nombre de lignes de connexion.
- Exemple 3 : si on veut minimiser le coût d'installation téléphonique dans un quartier, le problème peut être modélisé sous forme de recherche d'un arbre de poids minimal. Idem pour un réseau électrique...

Il existe plusieurs algorithmes pour trouver l'arbre de couverture de poids minimal : les plus populaires sont Kruskal et Prim.

3.7. L'algorithme de Kruskal

L'algorithme de Kruskal permet de trouver l'ACM d'un graphe pondéré.

Soient :

- $n = \text{ordre}(G)$;
- $m = \text{taille}(G)$; et
- T = l'ensemble des arêtes qui appartiennent à l'arbre cherché

Trier les arêtes a de G dans l'ordre croissant des poids

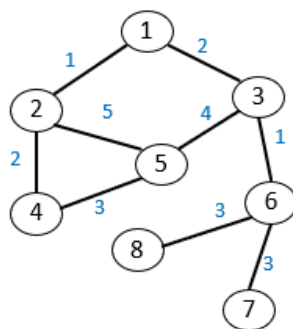
$T = \emptyset$; $k=1$;

Tant que $k \leq m$ et $\text{card}(T) < n-1$ faire

Si l'ajout de a_k ne crée pas de cycle alors $T = T \cup \{a_k\}$;

$k=k+1$;

Appliquons cet algorithme sur le graphe $G = (V, E)$ pondéré suivant pour trouver son ACM



En 1^{er} lieu, les sommets doivent être triés dans l'ordre croissant de leurs degrés, ce qui donne le tableau suivant :

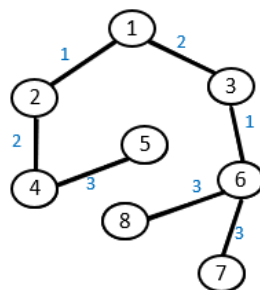
Arête	(1,2)	(3,6)	(1,3)	(2,4)	(4,5)	(6,7)	(6,8)	(5,3)	(2,5)
Poids	1	1	2	2	3	3	3	4	5
Indice k	1	2	3	4	5	6	7	8	9

Au démarrage $T = \emptyset$

Condition d'arrêt : $k \leq 9$ et nombre d'arêtes de l'arbre = $8-1 = 7$

k	L'ensemble F des arêtes de l'arbre	Action/remarque sur l'arête
1	$T = \{(1,2)\}$	L'arête (1,2) est ajoutée à l'arbre
2	$T = \{(1,2), (3,6)\}$	L'arête (3,6) est ajoutée à l'arbre
3	$T = \{(1,2), (3,6), (1,3)\}$	L'arête (1,3) est ajoutée à l'arbre
4	$T = \{(1,2), (3,6), (1,3), (2,4)\}$	L'arête (2,4) est ajoutée à l'arbre
5	$T = \{(1,2), (3,6), (1,3), (2,4), (4,5)\}$	L'arête (4,5) est ajoutée à l'arbre
6	$T = \{(1,2), (3,6), (1,3), (2,4), (4,5), (6,7)\}$	L'arête (6,7) est ajoutée à l'arbre
7	$T = \{(1,2), (3,6), (1,3), (2,4), (4,5), (6,7), (6,8)\}$	L'arête (6,8) est ajoutée à l'arbre
	A ce niveau, nous avons inséré 7 arêtes dans T et l'algorithme doit s'arrêter, car nous savons que $\text{Card}(T) = n-1 = 7$ dans le cas de ce graphe. Et de toute façon, si on tente de continuer les itérations, nous obtenons :	
8	$T = \{(1,2), (3,6), (1,3), (2,4), (4,5), (6,7), (6,8)\}$	L'arête (5,3) est ignorée car elle forme un cycle
9	$T = \{(1,2), (3,6), (1,3), (2,4), (4,5), (6,7), (6,8)\}$	L'arête (2,5) est ignorée car elle forme un cycle

Donc le résultat de l'algorithme est l'ACM suivant de poids total égal à 15.



Remarque : L'algorithme de Kruskal s'applique également sur un graphe non-connexe, et produit dans ce cas une forêt d'ACM. Le nombre des ACMs trouvés est le même que les composantes connexes du graphe original.